



TECHNICAL REFERENCE

# FocusMaxxer

Architecture & Domain Reference

---

Study-oriented technical documentation of the whole application: what it does, the scientific models it rests on, how it is engineered, and how it runs. Every formula, constant and threshold is traceable to a file:line reference in the source.

---

**SCOPE** SAFTE fatigue model · biometric stress analysis · session rules engine · Flutter/Provider architecture

**GENERATED** 2026-07-09

Study-oriented technical documentation of the whole application: what it does, the scientific models it rests on, how it is engineered, and how it runs. Every formula, constant and threshold is transcribed from the source and is traceable to a `file:line` reference.

**How to read this document.** Each section opens at the *conceptual* level (plain language) and then descends to the *technical* level (formulas, types, invariants). Slide preparation can stop at the first level; oral-exam defense uses the second. Cross-references replace duplication — a concept is defined once and linked thereafter.

**Symbol table** — core symbols used across sections. Secondary symbols ( $f$ ,  $a_s$ ,  $I_{max}$ ,  $r$ ,  $d$ ,  $h$ ,  $N$ ,  $s$ ,  $k$ , ...) are defined inline where they first appear.

| SYMBOL                      | MEANING                             | UNIT                                |
|-----------------------------|-------------------------------------|-------------------------------------|
| $t$                         | Decimal hour-of-day                 | hours [0, 24)                       |
| $R(t)$                      | Homeostatic cognitive reservoir     | reservoir units                     |
| $R_{wake}$                  | Reservoir level at the last wake-up | units                               |
| $R_c$                       | Reservoir capacity (full tank)      | units (2880)                        |
| $p$                         | Depletion rate while awake          | units/min (0.5)                     |
| $C(t)$                      | Circadian modulator                 | dimensionless $\approx [-1.5, 1.5]$ |
| $E$                         | Effectiveness / <b>Readiness</b>    | percent [0, 100]                    |
| $\Pi$                       | Sleep-inertia penalty               | effectiveness points                |
| $\eta$                      | Normalized sleep efficiency         | [0, 1]                              |
| $\mu_{base}, \sigma_{base}$ | HR baseline mean / std-dev          | bpm                                 |
| $z$                         | Z-score of a heart-rate sample      | dimensionless                       |
| $S$                         | Stress index                        | [0, 1]                              |

# Table of Contents

---

## PART I — PROBLEM & SOLUTION

|                            |   |
|----------------------------|---|
| 1. Vision and domain ..... | 4 |
|----------------------------|---|

## PART II — THE SCIENTIFIC MODELS (THE CORE)

|                                   |    |
|-----------------------------------|----|
| 2. The SAFTE fatigue model .....  | 6  |
| 3. Biometric analysis .....       | 9  |
| 4. The session rules engine ..... | 11 |

## PART III — SOFTWARE ARCHITECTURE

|                                                     |    |
|-----------------------------------------------------|----|
| 5. Layered structure and dependency injection ..... | 14 |
| 6. The time model .....                             | 16 |
| 7. Data sources .....                               | 17 |

## PART IV — EXECUTION

|                                         |    |
|-----------------------------------------|----|
| 8. The central state machine .....      | 20 |
| 9. End-to-end lifecycle .....           | 23 |
| 10. Persistence and consolidation ..... | 25 |

## PART V — SURROUNDING CONCERNS

|                                       |    |
|---------------------------------------|----|
| 11. UI and presentation .....         | 27 |
| 12. Security and technical debt ..... | 30 |

## APPENDICES

|                                                              |    |
|--------------------------------------------------------------|----|
| Appendix A — File → responsibility map .....                 | 32 |
| Appendix B — Domain constants (single source of truth) ..... | 33 |
| Appendix C — Diagrams .....                                  | 34 |
| Appendix D — Build, run & test .....                         | 35 |
| Appendix E — References .....                                | 36 |

## PART I

# Problem & solution

## SECTION 1

## Vision and domain

### ■ 1.1 Concept

FocusMaxxer is a Flutter application for **deep-work sessions**. Its thesis: the *right* moment and *right* length for focused work are not fixed (as in a plain Pomodoro timer) but depend on the user's **current cognitive readiness**. The app estimates that readiness by fusing two independent signals and turns it into a concrete recommendation — *focus now for N minutes, then rest for M*, or *do not start, you are too fatigued*.

The two signals:

- **Sleep-driven fatigue** — how rested the brain is, from a biomathematical fatigue model (**SAFTE**) fed by real sleep data.
- **Biometric stress** — how strained the body is *right now*, derived statistically from heart-rate (HR) and step streams.

### ■ 1.2 The two data worlds

A defining architectural fact: the two signals come from **different worlds**.

| SIGNAL        | SOURCE                 | TRANSPORT                           | WHEN                           |
|---------------|------------------------|-------------------------------------|--------------------------------|
| Sleep / SAFTE | IMPACT sleep backend   | HTTP + JWT (real network)           | Fetches <b>once at boot</b>    |
| HR + steps    | Local <b>simulator</b> | In-process function of virtual time | <b>Every tick</b> , no network |

The HR/steps simulator stands in for a wearable: there is no real sensor in this project, so biometrics are generated deterministically from virtual time (see §7.2). Sleep, by contrast, is a genuine authenticated REST call (§7.1).

## ■ 1.3 Domain glossary

| TERM                                          | DEFINITION                                                                                                    |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>SAFTE</b>                                  | <i>Sleep, Activity, Fatigue and Task Effectiveness</i> — the fatigue model computing readiness.               |
| <b>Reservoir</b> $R(t)$                       | Homeostatic cognitive reserve; depletes awake, refills asleep.                                                |
| <b>Circadian</b> $C(t)$                       | 24h (+12h) rhythm modulating alertness by time-of-day.                                                        |
| <b>Effectiveness / Readiness</b>              | The 0–100% score shown in the UI, assembled from $R$ , $C$ and sleep inertia.                                 |
| <b>Sleep inertia</b>                          | Post-wake grogginess; a decaying penalty on effectiveness.                                                    |
| <b>Baseline</b> $(\mu_{base}, \sigma_{base})$ | The user's calm resting HR statistics, captured at the start of a session.                                    |
| <b>Stress index</b> $S$                       | 0–1 measure of sustained HR overload via a Z-score density rule.                                              |
| <b>AFK</b>                                    | <i>Away-From-Keyboard</i> — user physically left (detected from steps) or app backgrounded.                   |
| <b>Off-protocol</b>                           | User keeps working past the recommended break; triggers escalating nudges then abort.                         |
| <b>Segment</b>                                | One focus block; followed by a break.                                                                         |
| <b>Vagal tone</b>                             | Parasympathetic (vagus-nerve) activity; its restoration is the physiological proxy for break recovery (§3.4). |
| <b>Wakelock</b>                               | OS hold that keeps the screen on; enabled by the engine during active session states.                         |
| <b>Tick</b>                                   | The engine's atomic time step: <b>5 s</b> of virtual time.                                                    |

## PART II

# The scientific models (the core)

These three sections describe the pure-domain layer ( `lib/functions/` ): no Flutter, no I/O, fully deterministic and testable. They are the intellectual object of the app; everything in Parts III–IV exists to feed and orchestrate them.

## SECTION 2

## The SAFTE fatigue model

Source: `safte_engine.dart` · Output model: `safte_state.dart`

### ■ 2.1 Concept

SAFTE tracks a cognitive **reservoir**  $R$  that drains linearly while awake and refills during sleep, all modulated by a circadian rhythm  $C(t)$ . From the reservoir level, the time-of-day, and a post-wake grogginess penalty it derives a single **effectiveness** score (0–100%) — the number the UI calls *Readiness*.

The engine is `static`-only and stateless; it is called in two moments:

1. **At boot**, to compute the reservoir level at the last wake-up ( `calculateCurrentWakeupReservoir` ), integrating the night's sleep.
2. **On demand**, to evaluate effectiveness at any instant ( `computeStateAt` ) — invoked on every engine tick and for future projections.

### ■ 2.2 The circadian modulator $C(t)$

Two summed harmonics: a primary ~24h rhythm and a half-amplitude 12h "post-lunch dip" wave.

$$C(t) = \cos\left(\frac{2\pi}{24}(t - 18)\right) + 0.5 \cdot \cos\left(\frac{2\pi}{12}(t - 21)\right)$$

| PARAMETER           | VALUE | MEANING                  |
|---------------------|-------|--------------------------|
| Primary period      | 24 h  | Main daily rhythm        |
| Primary phase       | 18.0  | Peak alertness ~18:00    |
| Secondary period    | 12 h  | Post-lunch dip wave      |
| Secondary phase     | 21.0  | —                        |
| Secondary amplitude | 0.5   | Half strength vs primary |

Implemented in `_computeCircadianModulator` (`safte_engine.dart:180`).

## ■ 2.3 Depletion while awake

The reservoir loses  $p = 0.5$  units per minute awake, floored at empty:

$$R(t) = \max(0, R_{wake} - p \cdot \Delta_{awake})$$

where  $\Delta_{awake}$  is minutes elapsed since wake-up. Capacity  $R_c = 2880$  units. At  $p = 0.5$  units/min, **one unit  $\approx$  2 minutes** of wakefulness, so a full reservoir drains in  $2880/0.5 = 5760$  min ( $\approx 96$ h) of continuous wakefulness.

**Caveat on the source comment.** `safte_engine.dart:21-25` describes the units as "1:1 to minutes" and the tank as "48h" — this wording is imprecise (it holds only if  $p = 1.0$ ). It stems from the literal 2880 doing double duty: it is both the reservoir capacity (in *units*) and, separately, `_maxPlausibleAwakeMinutes` = 2880 *minutes* = 48h (the awake-span sanity cap in §2.4). The two are distinct quantities that happen to share a value. No downstream number is affected: every effectiveness term uses only the ratio  $r = R/R_c$ .

## ■ 2.4 Reservoir at bedtime

Before integrating a night, the engine finds where the reservoir stood at bedtime

(`_reservoirAtBedtime`, `safte_engine.dart:102`):

$$R_{bed} = \max(0, R_{wake}^{prev} - p \cdot m_{awake})$$

with **robust fallbacks** to a standard-day value  $R_{bed}^{std} = R_c - p \cdot 960 = 2400$  (a healthy 16h awake span) when:

- there is no stored history (first boot), or
- the awake span is implausible ( $m_{awake} > 2880$  min = 48h, or negative from clock skew).

## ■ 2.5 Replenishment during sleep

Sleep is integrated **minute by minute** across actual time in bed ( `_replenishDuringSleep`, `safte_engine.dart:134` ), keeping  $C(t)$  aligned to real wall-clock time. Let  $\eta = \text{sleepEfficiency}/100$ . For each minute  $m$  (hour-of-day  $t_m$ ):

$$I_m = \text{clamp} \left( \underbrace{f(R_c - R)}_{\text{debt drive}} - \underbrace{a_s C(t_m)}_{\text{circadian sleep pressure}}, 0, I_{max} \right)$$

$$R \leftarrow \min(R_c, R + I_m \cdot \eta)$$

| PARAMETER                               | SYMBOL    | VALUE    |
|-----------------------------------------|-----------|----------|
| Sleep-debt repayment factor             | $f$       | 0.00312  |
| Circadian sleep weight                  | $a_s$     | 0.55     |
| Max sleep intensity (physiological cap) | $I_{max}$ | 3.4 /min |

Interpretation: the emptier the reservoir, the stronger the debt drive; the circadian term makes night hours restore more than day hours; efficiency scales how much of each minute actually restores. A malformed span (non-positive, or  $> 24\text{h}$ ) skips integration and returns  $R_{bed}$  untouched.

## ■ 2.6 Effectiveness (Readiness)

At an arbitrary target instant ( `computeStateAt`, `safte_engine.dart:198` ), with  $\Delta$  minutes awake,  $h = \Delta/60$  hours awake:

Reservoir ratio and depletion ratio:

$$r = \frac{R(t)}{R_c}, \quad d = 1 - r = \frac{R_c - R(t)}{R_c}$$

**Sleep-inertia penalty** — negative, decays exponentially with hours awake, amplified when the reservoir is low (grogginess lingers when already fatigued):

$$\Pi = -10 \cdot e^{-2h} \cdot (1 + d)$$

**Final effectiveness** — reservoir score + fatigue-amplified circadian swing + inertia penalty, clamped to a valid range:

$$E = \text{clamp} \left( 100r + C(t)(7 + 5d) + \Pi, 0, 100 \right)$$



| TERM                     | CONSTANT | ROLE                                                          |
|--------------------------|----------|---------------------------------------------------------------|
| Reservoir weight         | 100      | Reservoir contributes 0–100 points linearly                   |
| Circadian base amplitude | 7        | ±7 pts swing at full rest                                     |
| Circadian fatigue gain   | 5        | Extra ±5 pts swing as reservoir empties (→ ±12 when depleted) |

The result is packaged as an immutable `SafteState { effectiveness, reservoir, circadianValue, timestamp }`.

## 2.7 Conceptual summary

- Readiness is **not** just "how rested": it blends reserve level, time-of-day, and post-wake grogginess.
- Fatigue **amplifies** the body's sensitivity to circadian time (the *5d* gain).
- The model is **defensive by design**: every history-dependent step has a sane fallback so a bad or missing night never crashes or produces nonsense.

## SECTION 3

# Biometric analysis

Source: `biometric_analyzer.dart` · Shared resolution: `app_constants.dart`

## 3.1 Concept

The analyzer turns a live stream of HR + step samples into two engine signals: a **stress index**  $S \in [0, 1]$  and **AFK** detection. It keeps small sliding windows of recent samples and derives everything statistically against a per-session **baseline** captured during the calm start of a session.

All windows are sized in **ticks**; a tick is `tickDurationSeconds = 5 s` (`app_constants.dart:14`), so window lengths convert to real time explicitly:

| WINDOW                        | TICKS | REAL SPAN | PURPOSE                                  |
|-------------------------------|-------|-----------|------------------------------------------|
| <code>_window10Min</code>     | 120   | 10 min    | Baseline search pool (first 10 min only) |
| <code>_window3Min</code>      | 36    | 3 min     | Stress-rule density window               |
| <code>_window1Min</code>      | 12    | 1 min     | Recovery check                           |
| <code>_stepsWindow1Min</code> | 12    | 1 min     | AFK step counting                        |

## ■ 3.2 Baseline calibration

**Goal:** find the user's calmest resting HR — their "deep flow" cluster — inside the first 10 minutes.

`optimizeBaseline` (`biometric_analyzer.dart:119`) slides a fixed 36-tick (3-min) window over the pool and, for each cluster, computes

$$\mu = \frac{1}{N} \sum_i hr_i, \quad \sigma = \sqrt{\frac{1}{N} \sum_i (hr_i - \mu)^2}, \quad N = 36$$

(population variance, not sample). It keeps the **lowest**- $\sigma$  cluster (the calmest, most representative period), adopting it only if calmer than any seen before. The baseline used in Z-scores applies a floor:

$$\sigma_{base} = \max(\sigma_{raw}, 3.0)$$

**Why the floor  $\sigma_{base} \geq 3.0$ :** a tiny  $\sigma$  would make the Z-score hypersensitive (division by a near-zero spread). The floor also makes an explicit "flat sensor" guard unnecessary — a disconnected/flat signal can never turn the Z-score pathological.

Baseline optimization runs at  $t = 180$  s (lock) and re-optimizes every 15 s up to the 600 s calibration window (see §8.4).

## ■ 3.3 Stress index (Z-score + m-out-of-n)

**Concept:** one high beat is noise; *sustained* elevation is stress. Each recent tick is scored, and the **density** of anomalous ticks becomes the index.

Per-tick Z-score over the 3-min window:

$$z_i = \frac{hr_i - \mu_{base}}{\sigma_{base}}$$

A tick is **acute overload** when  $z_i \geq 2.0$  (i.e. HR at  $\geq +2\sigma$ ). The index is the anomalous fraction, saturating at 25 of 36 ticks (~70%):

$$S = \text{clamp}\left(\frac{\#\{i : z_i \geq 2.0\}}{25}, 0, 1\right)$$

**Acute overload** (the fail-safe that forces a break) fires when  $S \geq 1$ .

**Suppression during calibration:**  $S = 0$  while data is missing, before the baseline exists, or while still inside the 600 s calibration window — a transient rise there is not a reliable stress signal (`biometric_analyzer.dart:155`).

| CONSTANT                               | VALUE | ROLE                             |
|----------------------------------------|-------|----------------------------------|
| <code>_acuteOverloadZScore</code>      | 2.0   | Per-tick overload threshold      |
| <code>_stressSaturationCount</code>    | 25    | m-of-n saturation ("25 of 36")   |
| <code>_defaultSigma</code>             | 3.0   | Z-score stability floor          |
| <code>_incompleteRecoveryZScore</code> | 1.0   | Break recovery threshold (below) |

### ■ 3.4 Recovery check (break phase)

During a break the engine asks whether the body has actually recovered. `isRecoveryIncomplete` (`biometric_analyzer.dart:181`) averages the last minute of HR and Z-scores that average:

$$z_{avg} = \frac{\bar{hr}_{1min} - \mu_{base}}{\sigma_{base}}, \quad \text{incomplete} \iff z_{avg} > 1.0$$

If incomplete at a break-target crossing, the break auto-extends (see §8.6).

### ■ 3.5 AFK from steps

$$\text{stepsLastMinute} = \sum \text{steps window}, \quad \text{AFK} \iff \text{stepsLastMinute} > 10$$

Evaluated once per minute by the engine. 10 steps/min is the walk-away threshold.

### ■ 3.6 Conceptual summary

- The baseline is **personal and per-session**, not a fixed number.
- Stress is a **density** of anomalies over 3 minutes, not an instantaneous value — robust to single spikes.
- The analyzer stays **silent during calibration** so it never reacts to an unsettled baseline.

## SECTION 4

# The session rules engine

Source: `session_rules_engine.dart`

## ■ 4.1 Concept

Given the current SAFTE readiness, this pure engine returns the ideal **focus/break durations** for the next segment ( `SegmentTargets{focusSeconds, breakSeconds}` ). The policy in one line: *critically low readiness → forced short recovery; otherwise the block length emerges from a self-consistent forward search over the projected readiness curve.*

## ■ 4.2 Readiness grades and durations

Durations are defined in `session_rules_engine.dart:49-57` :

| GRADE                            | READINESS $E$    | FOCUS                | BREAK       |
|----------------------------------|------------------|----------------------|-------------|
| <b>Inhibited</b> (clinical lock) | $E < 65$         | 15 min (forced)      | 10 min      |
| <b>Warning floor</b>             | $65 \leq E < 77$ | 25 min               | 10 min      |
| <b>Interpolated</b>              | $77 \leq E < 90$ | 25 → 52 min (linear) | 10 → 17 min |
| <b>Optimal</b>                   | $E \geq 90$      | 52 min (cap)         | 17 min      |

The 52/17 pair is the classic deep-work rhythm.

The **inhibited** grade is normally unreachable (defensive path). The engine's clinical lock triggers at  $E < 65$ , but the UI's START gate only lets a session *begin* at  $E \geq 77$  ( `home_tab.dart:527-531` , via `SafteSemanticInterpreter.warningThreshold` ). Since  $77 > 65$ , `startSession()` never observes  $E < 65$  in the normal flow — readiness below 77 is already blocked at the button with a "READINESS TOO LOW" message ( `home_tab.dart:495-502` ). The **inhibited** state (and its Stage-1 lock below) is therefore **vestigial / defensive**: reachable only if `startSession` were invoked from another entry point. It is kept as a safety net, not dead code.

## ■ 4.3 Stage 1 — clinical lock

$$E < 65 \Rightarrow (\text{focus}, \text{break}) = (15, 10) \text{ min}$$

A short attempt followed by mandatory rest; readiness is too low to focus safely ( `calculateNextSegment` Stage 1, `session_rules_engine.dart:71` ). See the §4.2 note on why this branch is normally unreachable.

## ■ 4.4 Stage 2 — adaptive length via self-consistent stop

**Concept.** How long you *should* focus depends on readiness — but readiness itself changes minute by minute (circadian drift, depletion). The engine resolves this circularity by walking the *projected* readiness curve forward and stopping the block at the first minute whose elapsed time meets the duration that *that minute's* readiness prescribes.

The momentary "how long right now" mapping ( `_prescribedFocusMinutes`, `session_rules_engine.dart:134` ) linearly interpolates readiness [77, 90] onto focus [25, 52], clamped and **rounded to the nearest integer minute** (the operator `lerpClampRound` maps to the code's `_lerpClampedRound`):

$$\Phi(E) = \text{lerpClampRound}(E; [77, 90] \rightarrow [25, 52])$$

The forward search ( `session_rules_engine.dart:89-103` ), for  $k = 1, 2, \dots, 52$  minutes ahead:

```
for k in 1..52:
    E_k      = SAFTE effectiveness projected at (now + k minutes)
    phi_k    = prescribedFocusMinutes(E_k)          #  $\Phi(E_k)$ , in [25, 52]
    if k >= phi_k:                                  # self-consistent crossing
        focusMinutes = k
        break
    else:
        focusMinutes = 52                          # readiness stayed optimal
```

**Why it is bounded (no explicit clamp needed):**  $\Phi_k \geq 25$  means the crossing cannot occur before minute 25;  $\Phi_k \leq 52$  means it always occurs by minute 52, which also bounds the loop. The block naturally **grows when readiness rises** and **shrinks when it falls**.

#### ■ 4.5 Stage 3 — daily budget clamp

$$\text{focusSeconds} = \min \left( \text{focusMinutes} \times 60, \underbrace{14400 - \text{accumulated}}_{\text{remaining daily budget}} \right)$$

The daily cap is `dailyMaxSeconds = 240 × 60 = 14400 s (4 hours, session_rules_engine.dart:34)`; this engine *owns* that constant — every other layer reads it from here.

#### ■ 4.6 Stage 4 — proportional break

The break scales linearly with the **actual** (post-clamp) focus length, so a longer effort earns a longer rest:

$$\text{breakMinutes} = \text{lerpClampRound}(\text{actualFocusMin}; [25, 52] \rightarrow [10, 17])$$

#### ■ 4.7 Conceptual summary

- Duration is **prescribed by biology**, not chosen by the user.
- The self-consistent search elegantly handles the "duration depends on readiness which depends on time" circularity in one pass.
- The daily 4h cap is a hard clinical ceiling threaded through every calculation.

## PART III

# Software architecture

## SECTION 5

## Layered structure and dependency injection

Source: `main.dart`

### 5.1 Stack

- **UI / state management:** Flutter + **Provider** ( `ChangeNotifier` ). Consumption via `context.watch` / `read` / `select` — never `Provider.of` .
- **Persistence: Floor** (SQLite ORM) for sessions; `shared_preferences` for flags, tokens, and biological anchors.
- **Domain logic:** isolated pure functions in `lib/functions/` .

### 5.2 Layer map

```

screens/ (+ tabs/)           UI – observe providers, render, dispatch intents
  | watch / read / select
  ▼
providers/                   ChangeNotifier state holders (coordinators)
  | call
  ▼
functions/  services/  database/
  (pure)    (I/O)      (Floor persistence)

```

Rule: a widget reads providers; a provider orchestrates pure functions, services and the repository; pure functions know nothing of Flutter.

### 5.3 Dependency-injection tiers

`main()` opens `shared_preferences` and the Floor DB *before the first frame*, then wires a `MultiProvider` **ordered by dependency tier** — each tier may only read providers declared above it:

```

Tier 1  Data & hardware (no deps)
      └─ SessionRepository(database)
      └─ DeviceHardwareService

Tier 2  Base state
      └─ AuthProvider(prefs)
      └─ ImpactApiService      ← ProxyProvider: wired to AuthProvider.logout()
      │                        via onSessionExpired
      └─ SafteProvider(prefs)
      └─ GlobalClockProvider(speed = 60x, tick = 5 s)

Tier 3  Dependent
      └─ AnalyticsProvider(prefs, SessionRepository)

Tier 4  Central engine
      └─ CognitiveEngineProvider(Safte, Clock, Analytics, Hardware, scenario)

```

`CognitiveEngineProvider` sits at the top because it is the convergence point: it needs the SAFTE state, the clock, the analytics ledger and the hardware adapter all at once.

The one cross-tier callback: `ImpactApiService.onSessionExpired → AuthProvider.logout()` so a server-rejected session forces a logout ( `main.dart:96` ). It is wired with a `ProxyProvider` — a Provider whose value is (re)built from other providers it depends on — which lets the API service reach `AuthProvider` without holding a hard reference to it.

## ■ 5.4 Naming conventions

| SUFFIX / DIR                                                             | MEANING                                  |
|--------------------------------------------------------------------------|------------------------------------------|
| <code>*Provider</code> in <code>providers/</code>                        | <code>ChangeNotifier</code> state holder |
| <code>*Service</code> in <code>services/</code>                          | I/O boundary (network, hardware)         |
| <code>*Engine</code> / <code>*Analyzer</code> in <code>functions/</code> | Pure domain logic                        |
| <code>models/</code>                                                     | Plain data / Floor entities              |

## ■ 5.5 Key concepts

- Four DI tiers; a provider may read only tiers **above** it, so the graph stays acyclic.
- Pure logic ( `functions/` ) never imports Flutter; providers orchestrate, screens observe.
- One deliberate cross-tier link ( `onSessionExpired → logout` ) keeps auth and networking decoupled.

## SECTION 6

## The time model

**Source:** `clock_provider.dart` · **Consumer:** `CognitiveEngineProvider._onGlobalTick`

This is the most original — and most easily misunderstood — technical concept in the app, so it gets its own section.

### 6.1 Why virtual time

A full day of SAFTE dynamics would take a real day to observe. Since there is no emulator in this project (physical device only), `GlobalClockProvider` runs a **virtual clock faster than wall time** so a multi-hour fatigue simulation plays out in minutes.

### 6.2 The clock

| QUANTITY             | VALUE                           | NOTE                                                                   |
|----------------------|---------------------------------|------------------------------------------------------------------------|
| Speed multiplier $s$ | 60×                             | 1 real second → 60 virtual seconds (runtime-adjustable)                |
| Virtual tick         | 5 s                             | = <code>tickDurationSeconds</code> ; fixed so engine math never drifts |
| Real timer interval  | $\frac{5000}{60} \approx 83$ ms | <code>Timer.periodic</code> ; clamped to $\geq 1$ ms                   |

Each real tick advances virtual time by 5 s and emits `notifyListeners()` (`clock_provider.dart:88`):

$$\text{realInterval} = \left\lfloor \frac{\text{virtualTickSeconds} \times 1000}{s} \right\rfloor \text{ ms}$$

`setSpeedMultiplier` rebuilds the timer at a new pace on the fly (used by the profile's developer tools).

### 6.3 Lifecycle awareness

The clock is `WidgetsBindingObserver`:

- **paused** (real backgrounding): stop the timer (no ticks while away), record the wall-clock instant.
- **resumed**: **fast-forward** the virtual clock by the virtual time that elapsed while away, then restart ticking, so the fatigue model reflects the real gap instead of freezing:

$$\text{virtualMissed} = \lfloor \text{realMissedSeconds} \times s \rfloor$$



## ■ 6.4 The tick loop and catch-up

`CognitiveEngineProvider._onGlobalTick` ( `cognitive_engine_provider.dart:264` ) subscribes to the clock. Because a resume can jump virtual time forward by a large delta, the loop **catches up** by replaying missed ticks:

```
delta      = clock.now - internalClock      (seconds)
missedTicks = delta ÷ tickDurationSeconds

if missedTicks > maxCatchupTicks (360 ≈ 30 virtual min):
  # Long absence → do NOT replay (would freeze UI / grow buffer unbounded)
  if calibrating:      hold anomaly overlay for user RESTART/ABORT
  elif focus|break:    void the session (appBackgrounded / offProtocol)
else:
  for each missed tick: internalClock += 5 s; _processTick()
  internalClock = clock.now      # absorb sub-tick remainder
  notifyListeners()
```

**Why cap catch-up at 360 ticks:** replaying every tick of a long absence would block the UI thread and, in `breakMode`, grow the telemetry buffer without bound. Past the cap the session is treated as invalid.

## ■ 6.5 Conceptual summary

- One clock, one resolution ( `tickDurationSeconds` ) shared by clock, engine and analyzer — so the math never drifts between components.
- Virtual time is honest across backgrounding (fast-forward on resume).
- The engine's `_internalClock` tracks the shared clock but the engine owns the catch-up policy that protects UI and memory.

## SECTION 7

# Data sources

## ■ 7.1 Sleep source — IMPACT REST backend

**Source:** `impact_api_service.dart` · **Auth state:** `auth_provider.dart`

The **only** network source in the app. It talks to the IMPACT sleep backend ( `https://impact.dei.unipd.it/bwthw/` ) with JWT authentication.

**Endpoints & auth:**

| CONCERN                      | ENDPOINT                                                  | METHOD       |
|------------------------------|-----------------------------------------------------------|--------------|
| Login (credentials → tokens) | <code>gate/v1/token/</code>                               | POST         |
| Token refresh                | <code>gate/v1/refresh/</code>                             | POST         |
| Sleep record                 | <code>data/v1/sleep/patients/{patient}/day/{date}/</code> | GET (Bearer) |

**Token handling** ( `requestProtectedGet`, `impact_api_service.dart:127` ) refreshes both: - **preemptively** — if the local access token is missing/expired (via `JwtDecoder`), and - **reactively** — if the server still answers `401`, retrying the call once.

**Outcome taxonomy** ( `AuthOutcome` ): `success` · `invalidCredentials` (401/403) · `networkError` (retryable, must *not* log out) · `serverError`. This lets a genuine auth rejection be told apart from a transient blip: only the former forces a logout ( `onSessionExpired` → `SessionExpiredException` ).

**Boot fetch** ( `fetchMorningBaseline` ): the backend serves data with a **2-day lag**, so the record is requested for *two days ago*. Parsing is deliberately lenient — any anomaly (non-200, malformed JSON, unexpected shape) falls back to `_getMockBaseline()` (a synthetic "good 4-hour-ago night") so the app **always boots**, even offline.

**Security note:** tokens live in `shared_preferences` in **plaintext**; the service never logs token values, only error messages (see §12).

## ■ 7.2 Biometric source — local simulator

**Source:** `simulator_service.dart`

HR (bpm) and steps are **not** read from a sensor — they are generated locally as a deterministic function of virtual time and a seedable RNG, so every run is reproducible. Each `SimulationScenario` is a curated storyline that drives a specific engine behavior on screen:

| SCENARIO                     | STORYLINE                  | EXERCISES                                                                                                                 |
|------------------------------|----------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>steadyFocus</code>     | Calm HR, full recovery     | Normal focus/break cycle                                                                                                  |
| <code>stressPeaks</code>     | Two post-baseline HR peaks | Stress index $S$ rises without saturating, then saturates ( $S \geq 1$ ) → overload fail-safe (ring color mapping: §11.4) |
| <code>partialRecovery</code> | Break HR high then settles | One incomplete-recovery warning + auto-extension                                                                          |
| <code>failedRecovery</code>  | Break HR never settles     | Extensions until max break → session end                                                                                  |
| <code>taskAbandonment</code> | User walks away (steps)    | AFK detection + auto-resume                                                                                               |

**Timing invariant:** anything that must read as "anomalous" (the stress peaks) is scheduled **after** the 600 s baseline window, because an anomaly is only meaningful once  $\mu_{base}/\sigma_{base}$  are settled. HR is emitted as `base + rand(spread)` resting noise; `spread` is kept small so sampled  $\sigma$  stays under the analyzer's 3.0 floor.

### ■ 7.3 Key concepts

- Sleep is the **only** network dependency; everything biometric is local and deterministic.
  - Both sources are fault-tolerant: sleep falls back to a mock, the simulator is reproducible.
  - Scenarios are curated to exercise specific engine paths; anomalies are scheduled *after* calibration.
-

## PART IV

# Execution

## SECTION 8

## The central state machine

**Source:** `cognitive_engine_provider.dart` · **States:** `engine_state.dart`

### 8.1 Concept

`CognitiveEngineProvider` is where the two data worlds **converge**. On every clock tick it reads simulated HR/steps and the SAFTE readiness, feeds them to the `BiometricAnalyzer` and `SessionRulesEngine`, updates the `EngineState`, and delegates side effects (wakelock, vibration) to the hardware adapter — while owning a **volatile session buffer** until a session is validated and persisted.

### 8.2 The states

```

idle —startSession→ analyzingBaseline    (readiness ≥ 65)
idle —startSession→ inhibited             (readiness < 65)
idle —startSession→ idle                  (daily cap already reached)

analyzingBaseline —after 3 min→ focus
analyzingBaseline —AFK/anomaly→ overlay held → user RESTART/ABORT → idle

focus —manual break→ breakMode
focus —daily cap hit→ dailyLimitReached —after 2 s→ sessionEnded
focus —off-protocol 15' / AFK timeout→ sessionEnded

breakMode —manual resume→ focus
breakMode —manual end / long absence→ sessionEnded

inhibited / sessionEnded —resetEngine→ idle    (also: any state → idle)

```

| STATE                          | MEANING                                              |
|--------------------------------|------------------------------------------------------|
| <code>idle</code>              | No session running                                   |
| <code>analyzingBaseline</code> | Capturing initial calm HR to set the baseline        |
| <code>focus</code>             | Active deep-work segment                             |
| <code>breakMode</code>         | Recovery segment                                     |
| <code>inhibited</code>         | Focus refused — readiness critically low             |
| <code>dailyLimitReached</code> | 4h cap hit — a "good" ending (lingers 2 s on screen) |
| <code>sessionEnded</code>      | Finished; awaiting reset                             |

### ■ 8.3 The tick pipeline ( `_processTick` )

Each replayed tick ( `cognitive_engine_provider.dart:366` ):

1. Advance the simulator; read HR + step delta.
2. If **not** in an AFK/anomaly state: feed the sample to the `BiometricAnalyzer` and record it into the `ActiveSessionBuffer`.
3. Every 60 s: evaluate steps → arm AFK if `stepsLastMinute > 10` (focus/baseline only; movement is normal during a break).
4. Dispatch to the per-state handler ( `_handleAnalyzingBaseline` / `_handleFocusMode` / `_handleBreakMode` ).

### ■ 8.4 Calibration and baseline

- Session starts in `analyzingBaseline`. At  $t = 180$  s the baseline is **locked** ( `optimizeBaseline` ) and the state moves to `focus`.
- In focus, the baseline keeps **re-optimizing every 15 s until 600 s** (the `calibrationWindowSeconds`).
- An interruption anywhere inside the 600 s window is a **calibration anomaly**: the overlay is *held* for an explicit user RESTART/ABORT rather than a plain timeout, because a compromised baseline would poison the whole session ( `isCalibrationAnomaly`, `cognitive_engine_provider.dart:174` ).

### ■ 8.5 Focus mode: break advice and off-protocol escalation

`_handleFocusMode` ( `cognitive_engine_provider.dart:436` ):

- **Predictive daily-limit check** — if accumulated + buffered focus  $\geq 14400$  s, trigger `dailyLimitReached`.
- **Break recommendation** fires when either the target segment length is reached *or* `isAcuteOverload()` (stress  $S \geq 1$ ) — the biometric fail-safe.

- **Off-protocol escalation** if the user keeps working past the advice:

| ELAPSED PAST ADVICE | ACTION                                      |
|---------------------|---------------------------------------------|
| 5 min               | Nudge: "Break overdue. Interrupt now."      |
| 10 min              | Stronger: "OFF-PROTOCOL: stop and recover." |
| 15 min              | <code>endSession(offProtocol)</code>        |

## ■ 8.6 Break mode and recovery

`_handleBreakMode` ( `cognitive_engine_provider.dart:505` ). At each break-target crossing it consults `isRecoveryIncomplete()` (  $z_{avg} > 1$  ):

- **Incomplete** and extensions left ( $< 3$ ): extend the break by **+5 min**, warn "Vagal tone altered. Break automatically extended."
- **Incomplete** and max (3) reached: stop extending, flag "max break reached".
- **Complete**: recommend focus, "Vagal tone restored. Ready for Deep Focus."

A **manual** break lasts  $\max(300\text{ s}, 0.33 \times \text{focusElapsed})$  (at least 5 min).

**Who ends the session on max break (a deliberate abstraction leak).** When the extension budget is exhausted the engine only *sets* `_isMaxBreakReached`; it does **not** itself end the session.

`BreakModePage` observes that flag and calls `endSession(TerminationReasons.neuralFatigue)`, then navigates to the report ( `break_mode_page.dart:94-110` ). This is the one place the "thin screen" model deliberately leaks: the terminal decision on unrecoverable fatigue is driven from the UI, and this path is the **sole producer** of the `NEURAL FATIGUE` termination reason (§10.4).

## ■ 8.7 AFK, backgrounding and the buffer

- **Two AFK reasons:** `movement` (steps) and `background` ( `didChangeAppLifecycleState → paused` ). Only real `paused` trips it — `inactive` (notification shade, app switcher) does not.
- While AFK is active the segment timer **freezes** and counts toward `afkTimeoutSeconds = 60`; on timeout the session ends with the reason that reflects the true cause ( `userMovement` / `appBackgrounded` ).
- **Wakelock policy:** the screen is kept awake **iff**  $\text{state} \in \{ \text{analyzingBaseline}, \text{focus}, \text{breakMode} \}$  **and** no AFK warning is active ( `_updateWakelock`, `cognitive_engine_provider.dart:248` ); it is released on AFK, `idle`, `sessionEnded` and dispose. Alerts are a double-buzz vibration with a `HapticFeedback` fallback ( `device_hardware_service.dart:21` ).
- The `ActiveSessionBuffer` ( `session_data.dart:67` ) accumulates focus seconds and a per-tick HR timeline. It is *volatile* — it becomes a persisted `CognitiveSession` only if validated (see §10).

## ■ 8.8 Constant reference (engine)

| CONSTANT                              | VALUE       | ROLE                                 |
|---------------------------------------|-------------|--------------------------------------|
| <code>afkTimeoutSeconds</code>        | 60          | AFK grace before session end         |
| <code>calibrationWindowSeconds</code> | 600         | Baseline re-optimization window      |
| <code>_baselineReadySeconds</code>    | 180         | Baseline lock → focus                |
| <code>_baselineRefreshSeconds</code>  | 15          | Re-optimize cadence                  |
| <code>_afkStepThreshold</code>        | 10          | Steps/min → walked away              |
| <code>maxCatchupTicks</code>          | 360         | ≈30 virtual min; long-absence cutoff |
| <code>_protocolWarn1/2/Abort</code>   | 300/600/900 | Off-protocol escalation              |
| <code>_breakDurationRatio</code>      | 0.33        | Manual break = 33% of focus          |
| <code>_breakExtensionSeconds</code>   | 300         | +5 min per extension                 |
| <code>_maxBreakExtensions</code>      | 3           | Extension budget                     |

## ■ 8.9 Key concepts

- The engine is the sole owner of session state; it converges SAFTE readiness and biometric stress each tick.
- Calibration (0–600 s) is protected: an interruption there demands an explicit RESTART/ABORT.
- Two fail-safes end/redirect a session — acute overload (recommends break) and off-protocol escalation (aborts).
- One deliberate exception to "engine owns termination": max-break `NEURAL FATIGUE` is finalized by the break screen (§8.6).

## SECTION 9

# End-to-end lifecycle

This section **walks** the machine from the user's perspective, tracing state through the layers.

## ■ 9.1 Boot sequence

`BootloaderScreen` ( `bootloader_screen.dart` ) runs once on first frame:

```

1. IDENTITY ROUTING      AuthProvider.status →
                          firstTime      → OnboardingPage
                          unauthenticated→ LoginPage
                          authenticated  → continue

2. FETCH TELEMETRY      api.fetchMorningBaseline()      (2-day-lagged sleep)

3. SYNC ENGINE          safte.syncWithServer(...)      → recompute reservoir

4. RESOLVE WORKLOAD     if new biological day: analytics.resetDailyWork()

5. SYSTEM READY        → HomeDashboard

```

Failure handling: `SessionExpiredException` → route to login (a dead session's Retry could never succeed); a transient error → "SYNC FAILED. RETRY?" screen.

## 9.2 Identity & anchors

- `AuthProvider` decides the initial phase synchronously from `shared_preferences` (onboarding takes precedence over login; a stored login is trusted only while the refresh token is unexpired).
- `SafteProvider` holds the persistent **biological anchors** (`t_wake`, `t_sleep`, `r_at_wake`) and, on `syncWithServer`, recomputes the reservoir for **every** sleep event (naps and main sleep alike). Only a **main sleep** returns `isNewBiologicalDay = true`, which resets the daily counter. Within a tick, `getStateAt` is **memoized** (a single-entry cache invalidated on sync) so the many widgets asking for the same instant compute the SAFTE model only once.
- **Server-lag alignment:** every SAFTE projection shifts the target time back by `serverLag = 2 days` so "now" in the model lines up with the data actually available (`safte_provider.dart:154`).

## 9.3 A running session

```

Dashboard —startSession→ analyzingBaseline
  | (daily cap? → return; readiness < 65 → inhibited)
  ▼
analyzingBaseline —180 s→ focus —(re-optimize baseline to 600 s)
  |
  |— target reached / acute overload → break recommended
  |— user ignores 15 min → sessionEnded (offProtocol)
  |— daily cap → dailyLimitReached → sessionEnded
  |
  ▼ (manual)
breakMode —recovery complete→ focus (next segment)
  |— recovery incomplete → extend (x3) → max break
  ▼
sessionEnded → commit if valid → SessionReport → reset → idle

```

The dashboard, focus and break screens are thin: they `watch` the engine and render its getters; user intents (`startSession`, `manualTransitionToBreak`, `resolveAfkWarning`, ...) are method calls back into the engine — with one deliberate exception, the UI-driven max-break termination (§8.6).



## ■ 9.4 Key concepts

- Boot resolves identity → sleep fetch → SAFTE sync → daily reset → dashboard, tolerating every failure mode.
- The biological anchors and the 2-day server lag align "now" in the model to the data actually available.
- A session threads state upward through the layers; screens only observe and dispatch intents.

## SECTION 10

# Persistence and consolidation

**Sources:** `session_repository.dart` · `analytics_provider.dart` · `session_data.dart` · `termination_reason.dart`

## ■ 10.1 What is stored

Two stores with different roles:

| STORE                            | BACKEND                                           | CONTENT                                                                                    |
|----------------------------------|---------------------------------------------------|--------------------------------------------------------------------------------------------|
| Session history                  | Floor (SQLite) via <code>SessionRepository</code> | One <code>CognitiveSession</code> row per validated session                                |
| Live counters / anchors / tokens | <code>shared_preferences</code>                   | <code>worked_seconds</code> , <code>t_wake/t_sleep/r_at_wake</code> , JWTs, profile, flags |

`CognitiveSession` (Floor `@entity`): `id`, `date`, `durationSeconds`, `endingEffectiveness`, `hrTimelineJson`, `terminationReason`.

The Floor stack is thin: `AppDatabase` ( `@Database(version: 1, entities: [CognitiveSession])`, `app_database.dart:10` ) exposes a single `SessionDao` with three operations — `findAllSessions()` ( `SELECT ... ORDER BY date DESC` ), `@insert`, `@delete` ( `session_dao.dart` ) — behind `SessionRepository`. Deletion is user-driven from the Analytics tab ( `AnalyticsProvider.deleteSession` → DAO, §11.7), the mirror of the commit path. `_loadSessionsHistory` clones the DAO result into a growable list because Floor may return a fixed-length one.

## ■ 10.2 Validation

The volatile `ActiveSessionBuffer` is persisted **only if** it accumulated more than **600 s** of focus ( `isValidated`, `session_data.dart:83` ); shorter attempts are discarded as noise. Only focus/baseline ticks count toward `durationSeconds`; break ticks are still captured in the HR timeline but not counted.

### 10.3 The commit pipeline (idempotent)

`_commitSessionIfValid` ( `cognitive_engine_provider.dart:562` ) is guarded by `_isCommitting` / `_sessionCommitted` so a buffer is persisted **at most once**, even if `endSession` races a `detached` lifecycle event (the OS tearing the app down, delivered fire-and-forget):

```
endSession(reason):
  state = sessionEnded          # set FIRST (sync) → blocks re-entry from ticks
  await _commitSessionIfValid() # buffer still alive, state frozen
  notifyListeners()             # navigation happens post-persist

commitValidatedSession(session): # AnalyticsProvider – NOT idempotent
  id = await repository.saveSession(session) # write FIRST
  dailyWorkedSeconds += durationSeconds      # bump counter only after success
  sessions.insert(0, session)                 # newest first
  saveWorkloadToDisk()
```

**Why write-then-count:** if `saveSession` throws, the counter stays untouched and a retry re-runs cleanly — the same session can never be double-counted.

### 10.4 Termination reasons (storage contract)

`TerminationReasons` string values are written **verbatim** into the DB column and later matched by the UI to pick a badge — they are **immutable wire values**, not display text (renaming one would orphan every historical row):

`MANUAL END` · `CLINICAL LIMIT REACHED` · `NEURAL FATIGUE` · `OFF PROTOCOL` · `USER MOVEMENT` · `APP BACKGROUNDED` .

Where each is produced: `CLINICAL LIMIT REACHED` (daily cap, §8.5), `OFF PROTOCOL` (15-min escalation / long absence, §8.5), `NEURAL FATIGUE` (max-break, UI-driven, §8.6), `USER MOVEMENT` / `APP BACKGROUNDED` (AFK timeout, §8.7), `MANUAL END` (explicit stop, the neutral default).

### 10.5 Key concepts

- Two stores: Floor (durable session history) and `shared_preferences` (live counters, anchors, tokens).
- A session is saved only if it exceeds 600 s of focus; the commit is idempotent and write-then-count.
- Termination-reason strings are an immutable storage contract, not display text.

PART V

# Surrounding concerns

SECTION 11

## UI and presentation

Sources: safte\_semantic\_interpreter.dart · screens/ , screens/tabs/ , utils/

Screens are **observers**: they watch / select provider state, render it, and dispatch user intents back as method calls. They hold no domain logic (the one deliberate exception — the UI-driven max-break termination — is documented in §8.6).

### 11.1 Readiness → UI mapping

SafteSemanticInterpreter is the single place that turns raw SAFTE numbers into labels and theme colors ( safte\_semantic\_interpreter.dart:20-31 ):

| READINESS <i>E</i> | LABEL       | COLOR ROLE |
|--------------------|-------------|------------|
| ≥ 90               | OPTIMAL     | primary    |
| [77, 90)           | BALANCED    | secondary  |
| < 77               | COMPROMISED | error      |

The two boundaries (90, 77) are imported from the rules engine's optimal / warning thresholds so the UI and the scheduler never disagree on what "optimal"/"balanced" mean. Note the UI intentionally **collapses everything below 77 into one band** ( COMPROMISED ): it does not surface the engine's separate 65 inhibited boundary, because the START gate already blocks below 77 (§4.2).

It also labels reservoir fill ( High / Draining / Depleted at 0.8/0.4 ratios), circadian phase ( Peak / Stable / Slump ), and sleep-inertia clearance ( Severe / Active / Fading / Cleared at 15/60/120 min awake).

### 11.2 Screen map & navigation

bootloader → login / onboarding → home\_dashboard (hosting home\_tab / analytics\_tab ) → focus\_mode ⇌ break\_mode → session\_report ; profile\_page is reached from the dashboard app-bar. Custom route transitions ( ImmersiveRoute / FadeRoute / SessionActiveRoute , in route\_transitions.dart ) give each hop a distinct feel — e.g. the quick SessionActiveRoute snap between the two live focus/break surfaces. First launch runs a 3-slide onboarding carousel ( onboarding\_page.dart ) that sets the completeOnboarding flag and routes to login.

## ■ 11.3 Dashboard & Home tab

`HomeDashboard` is a two-tab shell: a `PageView` (Home / Analytics) with **swipe disabled** (tab changes only via the bottom `NavigationBar`) and `PageStorageKey`s that preserve each tab's scroll position (`home_dashboard.dart`).

The **Home tab** (`home_tab.dart`) renders: - the animated **readiness ring** (a `TweenAnimationBuilder` arc + the numeric score/label); - the **daily-workload bar**, whose maximum is sourced from `SessionRulesEngine.dailyMaxSeconds` so it can never drift from the enforced cap; - the **SAFTE components** tiles (reservoir / circadian / inertia, labeled via `SafteSemanticInterpreter`); - the bottom-pinned **START button**, in one of three mutually exclusive states (`_StartButtonStyle.resolve`): `LIMIT REACHED` (daily cap, outranks all), `READINESS TOO LOW` (gate  $E < 77$ ), or `START SESSION` (ready). Only the ready state calls `engine.startSession()` and opens the focus surface; the other two explain the block.

**Rebuild throttle (a notable performance mechanism).** The readiness widgets use `context.select` on the clock and key on the **floored** score / **bucketed** status labels (`home_tab.dart:175-178, 311-333`), so the many sub-integer SAFTE updates per second collapse into rare rebuilds (only when the displayed value actually changes).

## ■ 11.4 Focus surface: the biometric ring

The focus screen centers on `BiometricRing` (`biometric_ring.dart`), a custom-painted gauge whose fill is the **segment progress** and whose color encodes state — this is the on-screen embodiment of the stress index  $S$  from §3.3:

- Base color follows the `EngineState` (calibration = tertiary, deep focus = primary, break = secondary, ...).
- **Stress nudge (focus only, after calibration):** the ring color lerps from its base toward **amber** proportionally to  $S$ , and **snaps to the error color** once  $S \geq 1$  (`biometric_ring.dart:126-135`). This is the "amber → red" behavior the scenarios in §7.2 refer to.
- The nudge is suppressed during calibration and outside focus (the baseline isn't yet a reliable reference).
- A slow "breathing" pulse repaints via a `RepaintBoundary` / `CustomPainter` so it never dirties the widget tree.

## ■ 11.5 Break surface

`BreakModePage` (`break_mode_page.dart`) is a guided-recovery screen: a **breathing pacer** (an 8 s inhale/exhale cycle with the live timer) and, when `hasIncompleteRecovery` is set, a "LOOK AWAY" advisory that slides in. Controls are protective: **END requires a long-press** (a tap only hints the gesture, so a session can't end by accident), **RESUME stays locked** ("WAITING") until the engine sets `isFocusRecommended`, `PopScope(canPop: false)` blocks the system back gesture, and both controls disable while an AFK overlay is up. This screen also finalizes the max-break `NEURAL FATIGUE` termination (§8.6).

## ■ 11.6 The session report

`SessionReportPage` ( `session_report.dart` ) serves **two flows** from the same widget: - **live debrief** — reads the just-finished session from the engine and, on close, calls `resetEngine()` and unwinds to the app root; - **history replay** — read-only, rendered from a stored session's `historicalTimeline` (from the Analytics tab); close simply pops.

`PopScope(canPop: isHistory)` enforces that a live debrief must be closed consciously (so the engine resets) while history is freely back-navigable. `_computeStats` reduces the HR timeline to two headline metrics: **average HR** and **recovery minutes** (counted from non-focus ticks). The chart ( `f1_chart` ) draws the HR line over a **fixed 40–160 bpm axis** (comparable across sessions), shades each tick's time band by phase (focus vs recovery), and overlays a dashed average-HR reference line with a per-point time/BPM tooltip. A corrupt or empty timeline degrades to a "No physiological data" placeholder rather than crashing.

## ■ 11.7 Analytics tab (history & delete)

`AnalyticsTab` lists stored sessions newest-first, each card tappable into the report's history mode and swipe/button **deletable** ( `AnalyticsProvider.deleteSession` ), the mirror of the commit path in §10.3), with an empty state when there is no history. Reading a card's `hrTimelineJson` is a **guarded decode**: a malformed payload degrades to an empty timeline instead of throwing.

## ■ 11.8 Profile & developer tools

`ProfilePage` (reached from the Home app-bar person icon) edits the profile identity and exposes purge/logout, built from the `settings_components.dart` kit. It also hosts the **developer tools** that make live demos possible: - **Simulation Speed** — a dropdown over presets `[1x, 2x, 10x, 60x]` that calls `GlobalClockProvider.setSpeedMultiplier`, so the presenter can slow the clock to watch the ring evolve or keep it fast to fast-forward a session. - **Active Scenario** — a dropdown over `SimulationScenario.values` that calls `CognitiveEngineProvider.updateScenario`, swapping the biometric storyline on the fly.

This is *how the scenarios in §7.2 are triggered on screen during the exam demo*.

## 11.9 Shared helpers (reuse, don't duplicate)

| HELPER                                         | PURPOSE                                                                                                 |
|------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <code>utils/duration_format.dart</code>        | <code>formatClock</code> (live timer), <code>formatHuman</code> (totals)                                |
| <code>functions/termination_reason.dart</code> | <code>TerminationReasons.*</code> storage contract                                                      |
| <code>utils/termination_badge.dart</code>      | <code>TerminationBadge.forReason</code> — report ↔ history consistency                                  |
| <code>utils/ambient_glow.dart</code>           | Static radial-gradient decorative blobs                                                                 |
| <code>utils/route_transitions.dart</code>      | <code>ImmersiveRoute</code> / <code>FadeRoute</code> / <code>SessionActiveRoute</code> page transitions |
| <code>utils/dashboard_helpers.dart</code>      | Barrel: semantic interpreter, routes, <code>PremiumSliverAppBar</code>                                  |

## 11.10 Key concepts

- Screens are thin observers; the readiness widgets throttle rebuilds via `context.select` on bucketed values.
- The biometric ring is the visual reading of the stress index  $S$  (amber drift → red at saturation).
- The report is one widget serving both a live debrief (resets the engine) and read-only history replay.
- Developer tools (speed, scenario) on the Profile screen are the live-demo control surface.

## SECTION 12

# Security and technical debt

## 12.1 Security posture

- **JWTs in plaintext** — `access` / `refresh` tokens are persisted in `shared_preferences` with no secure storage. Migration to an (encrypted) Floor DB is planned. Mitigations in place: tokens are validated against expiry, a dead refresh token forces re-login, and **token values are never logged**.
- **Credentials** entered at runtime in login; only the JWTs are persisted.

## 12.2 Accepted debt (out of scope by decision)

Re-evaluable if requirements change: - `sleepEfficiency` clamp in `saft_engine`. - Single-flight on `refreshTokens` in `impact_api_service` (single-flight = collapsing concurrent refresh calls into one in-flight request). - Defensive `hr` casts in `session_report`. - `hrTimeline` cap in `session_data`.

### ■ 12.3 Maintenance notes

- Never print/log tokens or credentials.
- Do not edit generated files ( `database/app_database.g.dart` ).
- Detailed remediation history lives in the git log, not in docs.

### ■ 12.4 Key concepts

- Security posture is pragmatic: plaintext JWTs, mitigated by expiry checks, forced re-login and no token logging.
  - Several sharp edges are *accepted debt* by explicit decision, re-evaluable if requirements change.
-

## APPENDIX A

# File → responsibility map

| FILE                                                  | LAYER    | RESPONSIBILITY                                                                                                           |
|-------------------------------------------------------|----------|--------------------------------------------------------------------------------------------------------------------------|
| <code>main.dart</code>                                | entry    | DI wiring, DB/prefs bootstrap                                                                                            |
| <code>app_constants.dart</code>                       | pure     | <code>tickDurationSeconds</code> (shared resolution)                                                                     |
| <code>functions/safte_engine.dart</code>              | pure     | SAFTE fatigue math                                                                                                       |
| <code>functions/biometric_analyzer.dart</code>        | pure     | Baseline, stress index, AFK                                                                                              |
| <code>functions/session_rules_engine.dart</code>      | pure     | Focus/break duration policy                                                                                              |
| <code>functions/termination_reason.dart</code>        | pure     | Termination storage contract                                                                                             |
| <code>providers/clock_provider.dart</code>            | provider | Virtual time source                                                                                                      |
| <code>providers/safte_provider.dart</code>            | provider | Biological anchors + SAFTE gateway                                                                                       |
| <code>providers/cognitive_engine_provider.dart</code> | provider | Central state machine                                                                                                    |
| <code>providers/analytics_provider.dart</code>        | provider | Daily counter + session ledger                                                                                           |
| <code>providers/auth_provider.dart</code>             | provider | Auth/onboarding phase, profile                                                                                           |
| <code>services/impact_api_service.dart</code>         | service  | Sleep backend + JWT auth                                                                                                 |
| <code>services/simulator_service.dart</code>          | service  | HR/steps simulator                                                                                                       |
| <code>services/device_hardware_service.dart</code>    | service  | Wakelock, vibration                                                                                                      |
| <code>database/session_repository.dart</code>         | data     | Repository over Floor DAO                                                                                                |
| <code>database/session_dao.dart</code>                | data     | Floor DAO: find / insert / delete queries                                                                                |
| <code>database/app_database.dart</code>               | data     | Floor <code>@Database</code> definition (v1)                                                                             |
| <code>models/*</code>                                 | model    | <code>SafteState</code> , <code>DailyBaseline</code> ,<br><code>CognitiveSession</code> , <code>EngineState</code> , ... |
| <code>screens/*</code> , <code>utils/*</code>         | UI       | Presentation + shared helpers                                                                                            |



## APPENDIX B

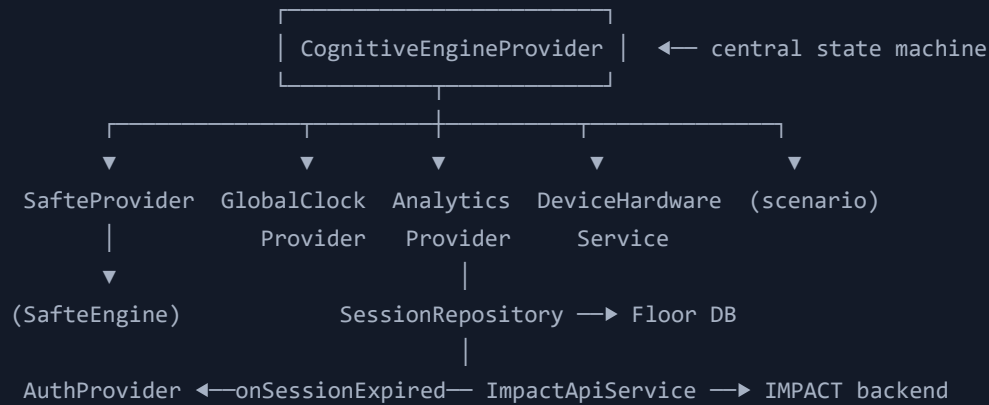
# Domain constants (single source of truth)

| CONSTANT                                | VALUE                  | OWNER                       |
|-----------------------------------------|------------------------|-----------------------------|
| Tick duration                           | 5 s                    | app_constants               |
| Clock speed                             | 60×                    | main (runtime-adjustable)   |
| Reservoir capacity $R_c$                | 2880                   | safte_engine                |
| Depletion $p$                           | 0.5 /min               | safte_engine                |
| Inhibited / warning / optimal readiness | 65 / 77 / 90 %         | session_rules_engine        |
| Optimal segment / break                 | 52 / 17 min            | session_rules_engine        |
| Warning segment / break                 | 25 / 10 min            | session_rules_engine        |
| Daily cap                               | 14400 s (4h)           | session_rules_engine        |
| Overload $Z$ / saturation               | $2.0\sigma$ / 25 of 36 | biometric_analyzer          |
| Baseline $\sigma$ floor                 | 3.0                    | biometric_analyzer          |
| Calibration window                      | 600 s                  | biometric_analyzer / engine |
| AFK step threshold / timeout            | 10 /min / 60 s         | engine                      |
| Server lag                              | 2 days                 | safte_provider              |

## APPENDIX C

# Diagrams

## ■ C.1 Dependency-injection graph



## ■ C.2 Tick sequence (per clock notify)

```

GlobalClock.notify
└─ Engine._onGlobalTick
    └─ delta / tick → missedTicks
    └─ if > 360: long-absence policy (hold | void)
    └─ for each missed tick: _processTick
        └─ simulator.getHR / getSteps
        └─ biometrics.addDataPoint + buffer.recordTick
        └─ AFK step check (per 60 s)
        └─ per-state handler (baseline | focus | break)
    └─ notifyListeners → UI rebuild
  
```

## ■ C.3 State machine — see [§8.2](#).

## APPENDIX D

# Build, run & test

---

**Toolchain & run** (physical device only — there is no emulator in this project):

| TASK                                                         | COMMAND                                                                                  |
|--------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Static analysis (target: zero issues)                        | <code>flutter analyze</code>                                                             |
| Run on a device                                              | <code>flutter run -d &lt;device-id&gt;</code> ( <code>flutter devices</code> for the id) |
| Floor codegen (regenerate <code>app_database.g.dart</code> ) | <code>dart run build_runner build --delete-conflicting-outputs</code>                    |

**Codegen.** The Floor entity/DAO/database annotations are compiled into the generated `part` file `database/app_database.g.dart` by `build_runner` ; that file is never edited by hand.

**Testing.** There is no automated test suite (only the default `test/widget_test.dart` ). The pure `functions/` layer is written to be trivially unit-testable (deterministic, Flutter-free), but tests are not currently provided; behavioral verification is manual on a physical device, exercising the full flow **login** → **boot** → **focus** → **break** → **report** → **analytics** plus a simulator scenario that triggers AFK / off-protocol. The network layer's `_getMockBaseline` fallback lets boot be tested without the backend.

## APPENDIX E

# References

---

- Hursh, S. R., et al. *Fatigue models for applied research in warfighting*. Aviation, Space, and Environmental Medicine, 2004 — the **SAFTE** model (reservoir, circadian process, sleep inertia).
  - Borbély, A. A. *A two-process model of sleep regulation*. Human Neurobiology, 1982 — the homeostatic (*R*) + circadian (*C*) decomposition SAFTE builds on.
  - Z-score / standard score — statistical normalization  $z = (x - \mu) / \sigma$ .
  - **m-out-of-n** detection rule — classical multi-sample decision logic (a detection fires when at least *m* of the last *n* samples are anomalous), here mapped to a graded index.
  - Vagal tone / heart-rate recovery — autonomic-recovery indicator informing the break-completion check.
- 

*This document is generated from the source at the referenced revision; when the code changes, update the affected section and its `file:line` anchors.*